



Alma Mater Studiorum-Università di Bologna

Scuola di Ingegneria

Fondamenti di Informatica T2

Esercitazione Individuale:

Array multidimensionali & ADT Matrix

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ROBERTA CALEGARI

Prof. AMBRA MOLESINI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



Agenda

- **FASE 1**

ripasso veloce array multidimensionali (matrici)

semplice esercitazione pratica con array multidimensionali

- **CASO DI STUDIO: matrici (libreria statica)**
- MA lavorare direttamente con gli array, esponendo direttamente la rappresentazione interna al cliente, può creare parecchi problemi..

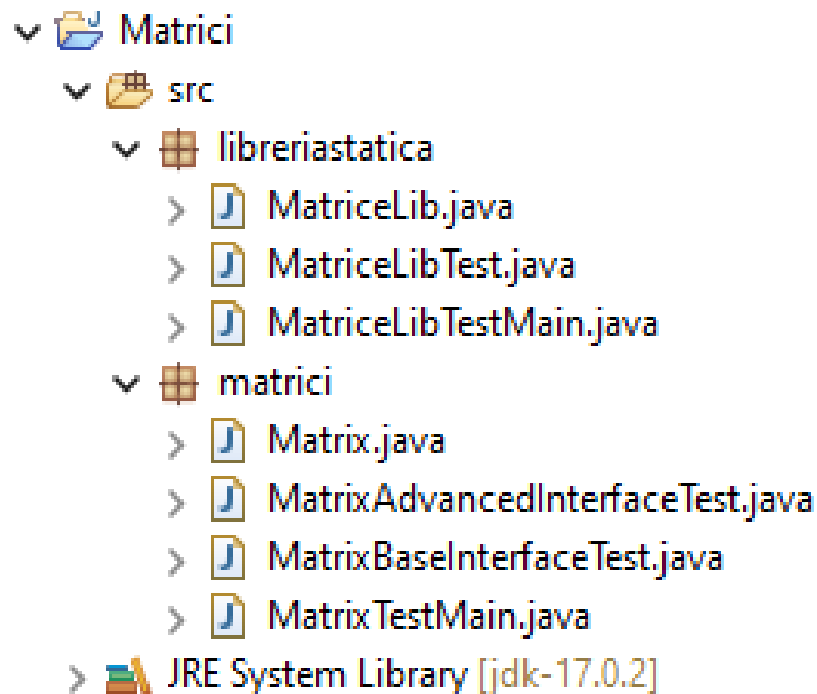
- **FASE 2 (revisione critica):**

progetto e realizzazione di un **nuovo ADT Matrix** che

- *incapsuli* al suo interno l'array (stato interno protetto, accesso mediato)
- esponga all'esterno un insieme di *metodi stabiliti da noi* (non da Java!)

Organizzazione dell'ambiente di lavoro

- Utilizziamo due diversi package
 - Package **LibreriaStatica**
 - contiene MatriceLib e le classi di test
 - Package **Matrici**
 - contiene ADT Matrix e le classi di test



Matrici come array multidimensionali

Una **matrice** è un **array a più dimensioni** (array di array).

Il *tipo del riferimento* specifica il *numero di dimensioni*:

```
double[][] m; // due dimensioni
```

La creazione avviene quindi in **due passi**:

- prima si crea l'**array "esterno"**:

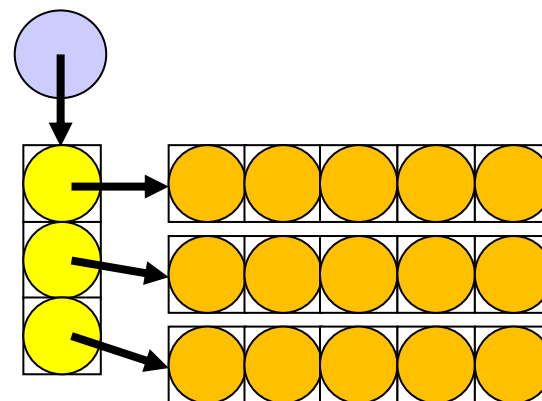
```
m = new double[3][];
```

- poi si creano **gli array "interni"**:

```
m[0] = new double[5];
```

```
m[1] = new double[5];
```

```
m[2] = new double[5];
```



Potremmo definire anche matrici *irregolari*, con righe di lunghezza diversa le une dalle altre.



Matrici come array multidimensionali

Una **matrice** è un **array a più dimensioni** (array di array).

Il *tipo del riferimento* specifica il *numero di dimensioni*:

```
double[][] m; // due dimensioni
```

La creazione avviene quindi in **due passi**:

- prima si crea l'**array “esterno”**:

```
m = new double[3][];
```

- poi si creano **gli array “interni”**:

```
m[0] = new double[5];
```

```
m[1] = new double[5];
```

```
m[2] = new double[5];
```

NB: siamo noi a decidere
come interpretarle in termini
di *righe* e *colonne*

Possiamo interpretare questa
matrice come 3x5
(3 righe x 5 colonne)
o viceversa: basta essere
coerenti nel seguito

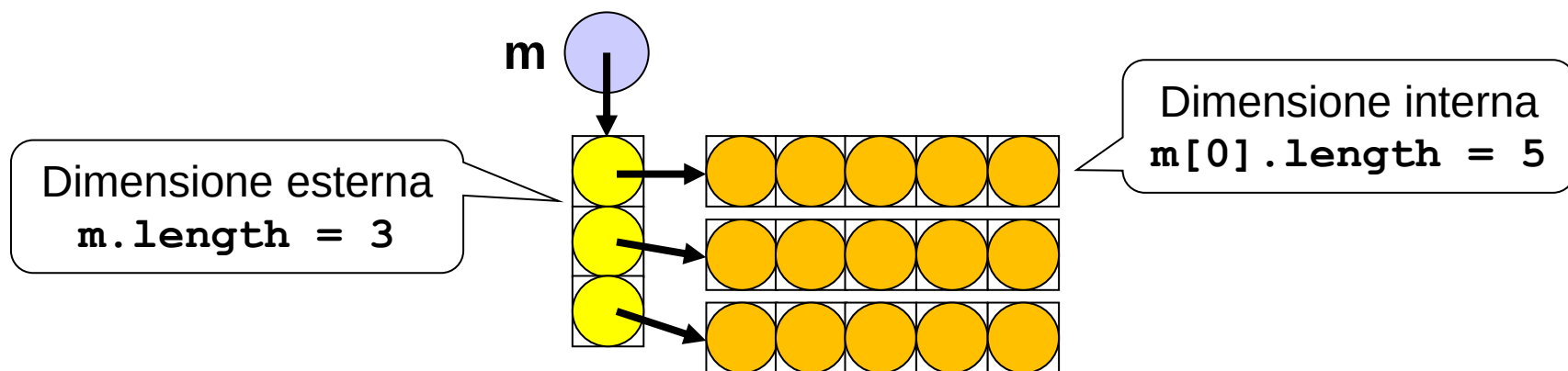
Array multidimensionali in Java

In Java, una **matrice regolare** si può creare più sinteticamente scrivendo:

```
m = new double[3][5];    // matrice 3 x 5
```

dove

- `m.length` è la cardinalità della **dimensione esterna**
- `m[i].length` è la cardinalità della **dimensione interna** (per matrici regolari, **i** può essere una qualsiasi)



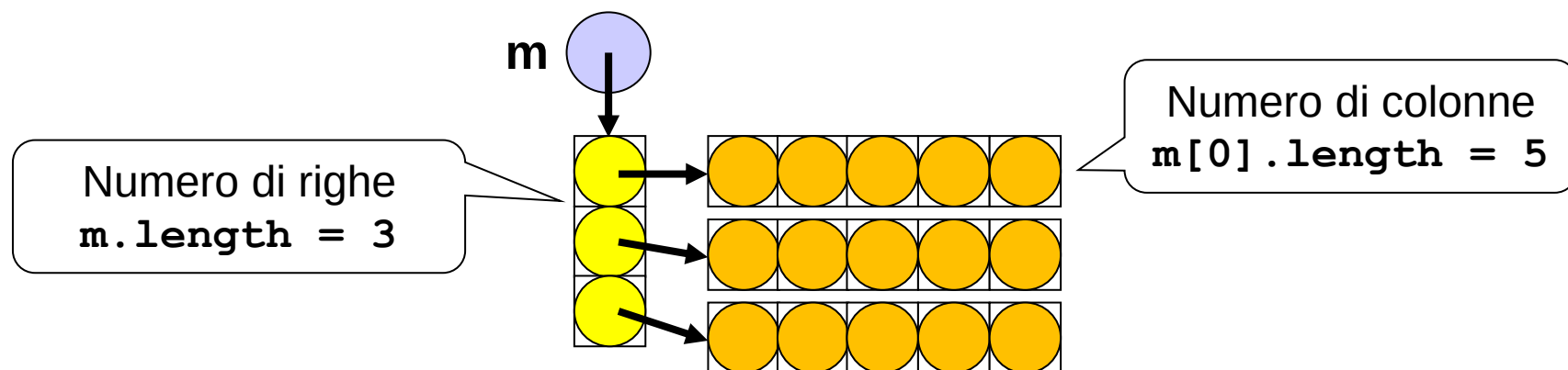
Array multidimensionali in Java

Per accedere alle celle si usano ovviamente *due indici*:

m[0][0] = 1.2; **m[1][0] = -2.6;** ...

la cui interpretazione in termini di *indice di riga* e *indice di colonna* dev'essere coerente con la precedente: qui,

- **m.length** è il numero di righe (quindi, **3**)
- **m[i].length** è il numero di colonne (quindi, **5**)





Esempio: Somma di matrici

```
public static double[][] sommaMatrici(  
    double[][] a, double[][] b) {  
    double[][] c = new double[a.length][a[0].length];  
    for (int i=0; i < a.length; i++)  
        for (int j=0; j < a[0].length; j++)  
            c[i][j] = a[i][j] + b[i][j];  
    return c;  
}
```

Perché una funzione statica?

- perché, se fosse un metodo, dovrebbe essere nella classe “array di array di **double**”, che *non* definiamo noi
- perciò non può che essere una funzione “classica” di libreria, che in Java assume la forma statica



Esempio: Somma di matrici

Mini-main di collaudo

```
public static void main(String[] args) {  
    double[][] m = {{ 1, 0, 0 }, { 0, 1, 0 },  
                    { 0, 0, 1 }};  
    double[][] n = {{ 2, 1, 1 }, { 1, 1, 0 },  
                    { 0, 0, 1 }};  
    double[][] q = sommaMatrici(m,n);  
    assert(3.0 == q[0][0]); // aggiungere altri controlli  
    stampaMatrice(q);      // da fare  
}
```

Esempio: Somma di matrici

Mini-main di collaudo

```
public static void main(String[] args) {  
    double[][] m = {{ 1, 0, 0 }, { 0, 0, 1 }, { 0, 1, 0 }};  
    double[][] n = {{ 2, 1, 1 }, { 1, 1, 0 }, { 0, 0, 1 }};  
    double[][] q = sommaMatrici(m,n);  
    assert(3.0 == q[0][0]); // aggiungere altri controlli  
    stampaMatrice(q);      // da fare  
}
```

Ma non sono double?
Perché usiamo degli
interi?

Il test di un numero double (o float) presenta alcuni problemi dovuti alla «precisione» del numero e facilmente portano al fallimento delle **assert**. Presto, adottando il framework Junit, risolveremo la problematica.



Esercizio: Prodotto di matrici

Analogo al precedente...

... MA *attenzione alle dimensioni delle matrici!*

Main di collaudo:

```
public static void main(String[] args){
    double[][] m = {{ 1, 0, 0 }, { 0, 1, 0 },
                    { 0, 0, 1 }};

    double[][] n = {{ 2, 1, 1 }, { 1, 1, 0 },
                    { 0, 0, 1 }};

    double[][] q = prodottoMatrici(m,n) ;
    assert(2.0 == q[0][0]) ;
    stampaMatrice(q) ;
}
```



Esercitazione: prima fase

- Implementare la classe **MatriceLib** con i relativi metodi statici
 - `public static double[][] sommaMatrici(double[][] a, double[][] b)`
 - `public static double[][] prodottoMatrici(double[][] a, double[][] b)`
 - `public static void stampaMatrice(double[][] a)`
- Nello startkit troverete già pronti **MatriceLibTest** per il collaudo della classe e **MatriceLibTestMain** per il collaudo della stampa

Tempo a disposizione: 30 minuti

Ricordatevi la direttiva **-ea** da dare alla JVM per far girare i test



Tutto bene, dunque?

- **Non proprio:**

- la soluzione adottata **espone direttamente la rappresentazione interna**: tutti vedono che la matrice è *fatta con array bidimensionali*
- è un approccio *error-prone*, che non protegge i preziosi dati:
chi ha in mano quell'array può farci letteralmente quello che vuole
 - perché non prendere $a[i,j]$ e non moltiplicarlo per pigreco?!?
- esporre la rappresentazione interna **NON È MAI UNA BUONA IDEA**

- **Revisione critica:**

- anziché esporre e manipolare direttamente l'array, *incapsularlo in un opportuno ADT (**Matrix**) che protegga lo stato interno*
- l'accesso ai dati non è più diretto, ma **intenzionalmente mediato**: la classe **Matrix** esporrà all'esterno *un insieme di metodi stabiliti da noi* che consentiranno solo le *manipolazioni sensate* dei dati

Verso l'ADT "Matrix"



ADT Matrix (1/8)

- OBIETTIVO: realizzare un componente Matrice (**Matrix**) come *tipo di dato astratto (ADT)*
- COSA VOGLIAMO come minimo (interfaccia di base):
 - *Costruire* una nuova matrice partendo dai valori che la matrice dovrà contenere
 - *Ottenere* il numero di righe e colonne della matrice
 - *Ottenere* il valore dell'elemento della matrice dati il numero di riga e di colonna
 - *Ottenere* una rappresentazione stringa della matrice
 - *Stabilire* se la matrice è quadrata



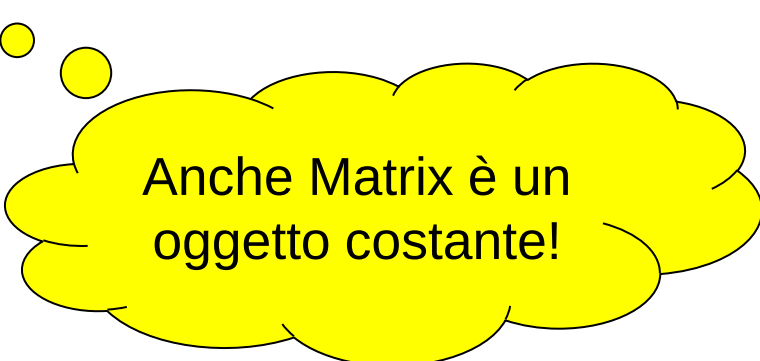
ADT Matrix (2/8)

- COSA VOGLIAMO in più (interfaccia estesa):
 - *Sommare* la matrice con un'altra e restituire una nuova matrice che rappresenti il risultato
 - *Moltiplicare* la matrice con un'altra e restituire una nuova matrice che rappresenti il risultato
 - *Ottenere* una nuova matrice che rappresenti una sottomatrice della matrice corrente, partendo da una riga e colonna di partenza e dal numero di righe e colonne da considerare per l'estrazione della sottomatrice
 - *Ottenere* una nuova matrice che rappresenti il minore della matrice corrente, partendo dalla riga e colonna da rimuovere
 - *Ottenere* il determinante della matrice

ADT Matrix (3/8)

E NATURALMENTE...

- Verificare il corretto funzionamento **di ogni singolo aspetto** del componente mediante opportuni test
 - Molti sono nello start kit, ma..
 - .. **alcuni dovreste farli VOI** 😊



Anche Matrix è un oggetto costante!

ADT Matrix (4/8)

Vista generale completa

Matrix
- values: double [][]
- calcDet() : double + det() : double + extractMinor(row :int, col :int) : Matrix + extractSubMatrix(startRow :int, startCol :int, rowCount :int, colCount :int) : Matrix + getCols() : int + getRows() : int + getValue(row :int, col :int) : double + isSquared() : boolean - Matrix(rows :int, cols :int) + Matrix(values :double[][]) + mul(m :Matrix) : Matrix - setValue(row :int, col :int, value :double) : void + sum(m :Matrix) : Matrix + toString() : String

ADT Matrix (5/8)

Vista generale completa

Matrix	
-	values: double [][]
-	calcDet() : double
+	det() : double
+	extractMinor(row :int, col :int) : Matrix
+	extractSubMatrix(startRow :int, startCol :int, endRow :int, endCol :int) : Matrix
+	getCols() : int
+	getRows() : int
+	getValue(row :int, col :int) : double
+	isSquared() : boolean
-	Matrix(rows :int, cols :int)
+	Matrix(values :double [][])
+	mul(m :Matrix) : Matrix
-	setValue(row :int, col :int, value :double) : void
+	sum(m :Matrix) : Matrix
+	toString() : String

PERCHÉ DUE COSTRUTTORI?

- Ipotesi: il cliente deve poter costruire solo matrici «piene», ben configurate: non devono esistere matrici «a metà»
- Il costruttore **pubblico Matrix/1** riceve perciò l'intero array bidimensionale con il contenuto

ADT Matrix (6/8)

Vista generale completa

```
- values: double ([][])  
  
- calcDet() : double  
+ det() : double  
+ extractMinor(row :int, col :int) : Matrix  
+ extractSubMatrix(startRow :int, startCol :int, endRow :int, endCol :int) : Matrix  
+ getCols() : int  
+ getRows() : int  
+ getValue(row :int, col :int) : double  
+ isSquared() : boolean  
- Matrix(rows :int, cols :int)  
+ Matrix(values :double[][])  
+ mul(m :Matrix) : Matrix  
- setValue(row :int, col :int, value :double)  
+ sum(m :Matrix) : Matrix  
+ toString() : String
```

Il costruttore ausiliario NON è indispensabile

- Si sarebbe potuto anche parcheggiare gli elementi via via calcolati in un array bidimensionale temporaneo, e solo alla fine incapsularlo in una nuova matrice usando il costruttore principale

PERCHÉ DUE COSTRUTTORI?

- Tuttavia, i metodi che calcolano e producono nuove matrici (sum, mul) devono internamente calcolarne gli elementi uno ad uno
- E' quindi utile un costruttore **privato ausiliario** che costruisca una matrice vuota, riservato a tali metodi

ADT Matrix (7/8)

Vista generale completa

Matrix	
-	values: double ([][])
-	calcDet() : double
+	det() : double
+	extractMinor(row :int, col :int) : Matrix
+	extractSubMatrix(startRow :int, startCol :int)
+	getCols() : int
+	getRows() : int
+	getValue(row :int, col :int) : double
+	isSquared() : boolean
-	Matrix(rows :int, cols :int)
+	Matrix(values :double[][])
+	mul(m :Matrix) : Matrix
-	setValue(row :int, col :int, value :double)
+	sum(m :Matrix) : Matrix
+	toString() : String

SCELTA DI PROGETTO

- il metodo **pubblico** `det()` effettua i controlli preliminari (matrice quadrata?), poi se tutto ok delega al metodo privato il calcolo effettivo
- Il metodo **privato** `calcDet()` incapsula tutta la logica di calcolo

PROBLEMA 1

- Cosa fa `det()` se la matrice non passa i controlli, ossia non è quadrata?



E se il risultato non c'è...?

- In molte situazioni ci si trova di fronte a **funzioni che dovrebbero restituire un valore che però non hanno**
 - esempio: calcolo del determinante, se la matrice non è quadrata
- Come comportarsi in questi casi...?
 - per i tipi-oggetto, si può **restituire null** (con tutti i rischi del caso)
 - MA **per i tipi primitivi...?**
- Per tamponare almeno parzialmente il problema, **le due classi di libreria Float e Double** incorporano *servizi di utilità* rispettivamente per i tipi primitivi **float** o **double**
- In particolare definiscono **la costante NaN** (*Not a Number*)

Not A Number

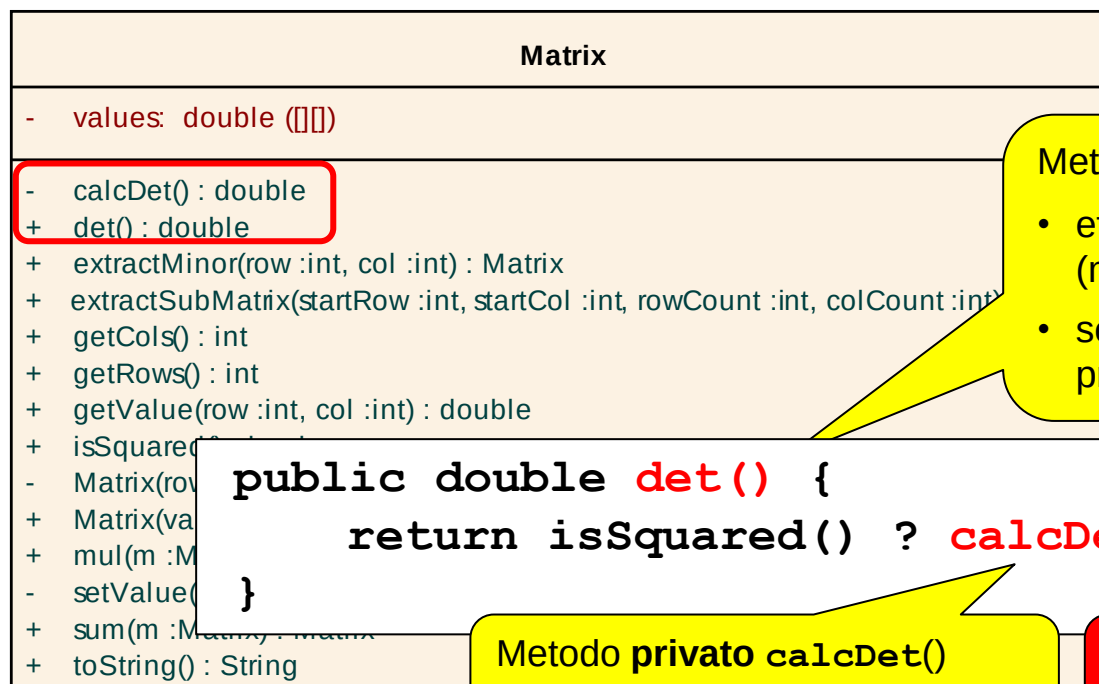
- Le due classi **Float** e **Double** sono una sorta di «libreria matematica» per i tipi primitivi **float** o **double**, rispettivamente
- Fra le altre cose, definiscono tre *costanti pubbliche*:

MAX_VALUE	A constant holding the largest positive finite value of type float, $(2 \cdot 2^{23}) \cdot 2^{127}$.
MIN_VALUE	A constant holding the smallest positive nonzero value of type float, 2^{-149} .
NaN	A constant holding a Not-a-Number (NaN) value of type float.
NEGATIVE_INFINITY	A constant holding the negative infinity of type float.
POSITIVE_INFINITY	A constant holding the positive infinity of type float.

- **NaN** è il valore restituito dalle forme indeterminate ($\infty - \infty$ o ∞ / ∞), o da *funzioni che non riescono a computare il risultato*

ADT Matrix (8/8)

- Quindi, se il determinante non si può calcolare perché la matrice non è quadrata, possiamo **restituire NaN**



```
public double det() {
    return isSquared() ? calcDet() : Double.NaN;
}
```

Metodo **pubblico** det ()

- effettua i controlli preliminari (matrice quadrata?)
- se tutto ok delega al metodo privato il calcolo effettivo

Metodo **privato** calcDet()

- incapsula la logica di calcolo

Se la matrice **NON** è quadrata

- si segnala errore restituendo NaN



Interfaccia di base (1/2)

- Stato:
 - scegliamo un *array bidimensionale* di **double**
- Costruttori:
 - **uno privato, a uso interno**
(due argomenti: numero di righe, numero di colonne)
 - **uno pubblico**
(un solo argomento: un array bidimensionale di **double** contenente tutti gli elementi che compongono la matrice)
 - **REQUISITO DI CONSISTENZA:** l'array ricevuto come argomento deve essere **ricopiato su quello interno valore per valore**, perché se si copiasse solo il riferimento qualcuno da fuori potrebbe cambiare lo stato interno dell'oggetto, violando l'incapsulamento → *pericoloso*
 - **MA** ciò è chiaramente oneroso: *si potrebbe evitare se la struttura dati fosse imm modificabile...*



Interfaccia di base (2/2)

- Accessor:
 - ottenere il numero di righe o colonne (**getRows** / **getCols**)
 - ottenere il valore di un particolare elemento della matrice (**getValue**)
 - *non vogliamo un metodo per impostare il valore di un elemento, perché anche la matrice, come una stringa, dev'essere un oggetto costante!*
 - una volta costruita, non deve essere alterata
 - potrà però esserci utile un *metodo privato* **setValue** a uso interno
- Ulteriori metodi per:
 - ottenere la rappresentazione in stringa del componente (**toString**)
 - stabilire se la matrice è quadrata (**isSquared**)



Riflessione sui costruttori

- *Chi crea l'array bidimensionale che conterrà i valori?*
 - i costruttori
- Più esattamente, *chi crea cosa* nei vari casi?
 - l'array bidimensionale va *allocato in entrambi i casi*
 - nel caso del *costruttore privato* con argomenti (righe, colonne), occorre creare un array bidimensionale *grande quanto indicato dai parametri*
 - nel caso del *costruttore pubblico* con argomento un array bidimensionale, occorre creare un array bidimensionale *grande come quello ricevuto*

Interfaccia estesa (1/4)

ULTERIORI METODI:

- **extractMinor** deve produrre e restituire una nuova matrice che rappresenti il *minore della matrice corrente*, che deve perciò essere *quadrata*
 - il metodo riceve in ingresso *gli indici della riga e della colonna* da sopprimere
 - restituisce una nuova matrice, identica all'attuale *ma PRIVA di tali riga e colonna*
- **det** calcola il *determinante* della matrice (quadrata) definito come:

$$\det(A_k) = \sum_{i=1}^n (-1)^{i+k} a_{i,k} \det(A_{i,k})$$

dove:

- $a_{i,k}$ è l'elemento di coordinate i,k
- $A_{i,k}$ è il minore ottenuto sopprimendo la i -esima e la k -esima colonna (ottenibile tramite **extractMinor**)

Interfaccia estesa (2/4)

ULTERIORI METODI:

- **extractSubMatrix** estrae dalla matrice corrente una nuova matrice che rappresenti una sua *sottomatrice*
 - riceve quattro argomenti: i primi due rappresentano gli indici di riga e colonna da cui iniziare l'estrazione...
 - ...mentre gli altri due costituiscono il numero di righe e di colonne che comporranno la sottomatrice
- **sum** produce una nuova matrice che sommi la matrice corrente a un'altra ricevuta come argomento
- **mul** produce una nuova matrice che moltiplichi la matrice corrente per un'altra ricevuta come argomento

Interfaccia estesa (3/4)

VERIFICA DELLE PRECONDIZIONI

- Il buon funzionamento di molti metodi dipende dal rispetto di specifiche *precondizioni*, che occorre preliminarmente verificare
- Problema: *che fare se qualche precondizione è violata?*
 - in astratto, bisognerebbe fermare l'esecuzione e segnalare il problema, *ma per ora non disponiamo degli strumenti adatti per farlo* ☹
 - quindi, *per ora ci accontenteremo di restituire **null**, pur sapendo che è largamente insoddisfacente*, in quanto:
 - non impedisce la prosecuzione della funzione, che perciò probabilmente causerà danni e forse l'esplosione completa del programma
 - non fa capire la causa prima del problema

Interfaccia estesa (4/4)

CASI CRITICI DA INTERCETTARE

- **extractMinor**
 - indici di riga/colonna «esterni» alla matrice corrente
 - matrice corrente *non quadrata*
- **extractSubMatrix**
 - indici di riga/colonna «esterni» alla matrice corrente
 - richieste *più righe/colonne di quelle disponibili*
- **sum**
 - matrici di dimensioni *non compatibili* con l'operazione $A + B$
(compatibilità: stesso numero di righe e di colonne in A e in B)
- **mul**
 - matrici di dimensioni *non compatibili* con l'operazione $A \times B$
(compatibilità: numero di colonne di A = numero di righe di B)

Interfaccia estesa: collaudo

- I test sono nello start kit!



I test verificano il funzionamento di *tutti* i metodi:

- *sia* nel caso di parametri corretti
- *sia* nei casi di parametri errati
- Se pensate che manchi qualche test... può essere 😊
Fatelo voi!



Esercitazione: seconda fase

- Implementare la classe **Matrix** con i relativi metodi
- Nello Startkit troverete già pronti:
 - **MatrixTestMain** contenente il main di collaudo
 - **MatrixBaseInterfaceTest** per collaudare l'interfaccia base
 - **MatrixAdvancedInterfaceTest** per collaudare l'interfaccia avanzata

Tempo a disposizione: 1 ora e 30 minuti

Ricordatevi la direttiva **-ea** da dare alla JVM per far girare i test